

CarHunter

CarHunter documentation · source: README.md · generated June 2026

CarHunter

A self-running web app that finds undervalued **private-party** cars on KSL Cars, scores every listing against a wholesale-profit buy-box, ranks the deals, and alerts you on the best ones — so you never open a scraper, a terminal, or a database. It scrapes on a schedule inside sandboxes, values each car against real market comps, prices recon on broken cars from real used-parts data, and surfaces the money in a ranked feed you work from a pipeline board.

Scope note (this build). Per the owner's direction, the seeded strategy is **KSL-only** and runs **broad market scans** (\$500–\$28k, all makes/years/titles) — the scoring engine is the arbitrage finder, not the search filter. Facebook Marketplace is architected-for but deferred (no credentials). The only hard filter upstream of you is **private-party-only** (dealer margin isn't your margin). Every mechanic special is surfaced for your manual review, never hidden by the math.

What it does

- 1 Your buy-box scans run **on a schedule** — you set up nothing.
- 2 The home feed is a **live, ranked list** of private-party cars: photo, year/make/model
- 3 trim, miles, asking price, **estimated value**, **estimated profit**, deal score (0–100), source badge, distance, days listed, and a link to the listing.
- 4 Any car with **estProfit** $\geq$ **\$1,500** is flagged **HOT** and pushed to you (email/SMS) the moment it lands — once per car unless its price drops.
- 5 **Pursue / Pass / Contacted** on each card; pursued cars move to a pipeline board (Lead → Contacted → Negotiating → Bought → Flipped → Dead) with suggested target-buy and walk-away numbers.
- 6 Price drops, relists, and gone/sold are tracked automatically; one row per real car (deduped per spec \$5).

Stack

- **Convex** — database, scheduled cron jobs, server functions, reactive queries, HTTP ingest
- **Daytona** — sandboxed runtime for the scrapers (one sandbox per run, torn down after)
- **Playwright / Python** — scraping (KSL uses its JSON API; Playwright rides along for the FB re-enable)
- **React + Vite + Tailwind** — the dashboard (mobile-first, dark dealer-tool theme)
- **TypeScript everywhere** except the Python scrapers

Prerequisites

- **Node 20+** and **Python 3.11+**
- A **Convex** deployment (Convex Cloud account, or the self-hosted local backend — see below)
- A **Daytona** account + API key (for scheduled scraping in production)

- Optional keys — the app degrades gracefully without each:
- `MARKETCHECK_KEY` — live retail comps without an operator. Without it, comps are seeded into the `comps` cache from MarketCheck queries; missing comps fall back to a flagged depreciation curve (amber, never HOT-alerted).
- `RESEND_KEY` — email alert delivery. Without it, alerts still log in-app.
- `TWILIO_ACCOUNT_SID` / `TWILIO_AUTH_TOKEN` / `TWILIO_FROM` — SMS alert delivery.

Setup (fresh clone)

```
git clone <repo> carhunter && cd carhunter
```

```
# 1. backend deps + Convex
npm install
npx convex dev          # provisions/links a deployment, pushes schema + functions
                        # (Convex Cloud writes CONVEX_DEPLOYMENT + VITE_CONVEX_URL
                        # into .env.local; self-hosted uses CONVEX_SELF_HOSTED_URL)

# 2. seed: settings (margin 1500, fees 400), the market scans, and ~1,226
# used-engine/transmission price keys from data/carpart_prices.csv
npm run seed

# 3. deployment env vars (Convex dashboard → Settings → Environment Variables,
# or `npx convex env set NAME value`). CONVEX_DEPLOYMENT is set for you by
# `npx convex dev` above; the rest you set:
npx convex env set INGEST_SECRET "$(openssl rand -hex 24)" # shared secret for /ingest
npx convex env set INGEST_URL "https://<your-deployment>.convex.site/ingest"
npx convex env set DAYTONA_API_KEY "dtn..." # production scraping
# optional: MARKETCHECK_KEY, RESEND_KEY, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_FROM

# 4. the dashboard
cd web
npm install
echo "VITE_CONVEX_URL=$(grep CONVEX_URL ../.env.local | cut -d= -f2)" > .env.local
npm run dev          # http://localhost:5173
```

Open the app, go to **Settings**, and paste your alert email/phone (and, when re-enabling Facebook, your FB **session cookie** — never a password, RULES #5).

Building the Daytona scraper image (first deploy)

Scrapers run only in Daytona sandboxes (never on the app server — RULES #6). Build a snapshot named `carhunter-scraper` with Python 3.11 + Playwright + the `scrapers/` folder baked in, then set `DAYTONA_API_KEY` + `INGEST_URL`. Once both env vars exist, `crons.ts` dispatches each due search into its own sandbox automatically.

Self-hosted / offline backend (this build)

Convex Cloud was unreachable in the build environment, so development used the self-hosted `convex-local-backend` binary:

```
SECRET=$(openssl rand -hex 32)
convex-local-backend --instance-name carhunter --instance-secret "$SECRET" \
  --port 3210 --site-proxy-port 3211 &
ADMIN_KEY=$(convex-local-backend keygen admin-key \
  --instance-name carhunter --instance-secret "$SECRET")
# .env.local: CONVEX_SELF_HOSTED_URL=http://127.0.0.1:3210
# CONVEX_SELF_HOSTED_ADMIN_KEY='<ADMIN_KEY>'
npx convex dev --once # pushes against the local backend
```

When `app.daytona.io` is unreachable (as in the build env), run scans in-container with the equivalent dispatcher instead of cloud sandboxes:

```
python3 scripts/dispatch_local.py --once      # one pass; --loop 60 to poll
```

It mirrors the production sandbox lifecycle step-for-step: per-search isolated process, POST to `/ingest`, `markRun`, `timeout-kill` teardown, concurrency cap 3.

Using it

Add a buy-box / scan — Searches tab → *New search*. Leave makes/models **empty** to scan the whole market (the default posture: never pre-filter a deal away). Set price band, radius, interval; toggle active. Each row shows last-run time, # new deals, and any error.

Read the feed — Feed tab. Cards are ranked hottest-first. Red **HOT** ribbon = $\text{estProfit} \geq \text{your threshold}$. Purple ■ **Special** = a drivetrain-issue car flagged for your review. Amber flags = *curve value* (no market comps yet — verify) or *keyword recon* (no used-parts data for that model). Filters narrow only when you ask. The **Specials** tab is your dedicated mechanic-special review queue.

Work the pipeline — Pursue a card → it lands in **Pipeline** as a Lead with suggested **target buy** and **walk-away** numbers. Drag between stages (or use the stage menu), add notes.

Detail drawer — tap any card for the full **profit math line by line**, the comp set used (source + sample size), the **recon math** broken out (used-part median, sample size, labor — so a mechanic special's risk is visible), price history, and the suggested numbers.

The scoring formula (so you can tune it — §4)

$\text{estProfit} = \text{estValue} - \text{price} - \text{estRecon} - \text{estFees}$

```
dealScore (0-100) =
  45 × clamp(estProfit / 4000, 0, 1)           # profit is king
+ 20 × clamp((estValue - price) / estValue, 0, 1) # margin %
+ 15 × freshnessBonus(daysListed)             # newer listing = more bonus
+ 10 × mileageFit(mileage, buyboxBand)
+ 10 × titleBonus(titleStatus)                # clean = full 10
```

$\text{hot} = \text{estProfit} \geq \text{marginThreshold}$ (default \$1,500) AND comp source $\neq$ curve

- **estValue** — comps come from MarketCheck first (sold comps beat asking comps), cached 7 days; otherwise a flagged depreciation curve (amber, never HOT). **Salvage/rebuilt = $0.70 \times \text{clean-title comp value}$** (clean comps only). Mileage $\pm \$0.06/\text{mi}$ vs the bucket midpoint; AWD/4WD +3%.
- **estRecon** — base \$400. Drivetrain failures are priced from **real used-part medians**: engine = median + \$1,300 labor, transmission = median + \$900; a generic "mechanic special" takes the **pricier** component (worst case protects margin). Models with no parts data fall back to the §4 keyword bumps (flagged). Other issues use the §4 bumps (salvage/rebuilt +\$2,500, doesn't-start +\$2,000, needs-work +\$1,000, accident +\$800, check-engine +\$600, new tires/brakes -\$200).
- **Suggested numbers** — `targetBuy = estValue - estRecon - estFees - margin`; `walkAway = ... - margin × 0.6`.

Change the margin threshold and flat fees in **Settings**.

Tests

```
npm run test:unit          # vitest - scoring/recon/dedupe/alerts/daytona (66 tests)
```

```
python3 -m pytest scrapers/tests/ # parser + KSL scraper (61 tests)
cd web && npm run build            # typecheck + production build
cd web && npx playwright test      # dashboard e2e (6 tests, needs the backend + a feed)
```

Project layout

```
convex/      schema, scoring, comps, recon, ingest (http.ts), crons, daytona, alerts, seed
  lib/       pure scoring math, recon rules, dedupe (sha1), depreciation curve, transports
scrapers/    ksl.py (JSON API), parse.py (normalizer + dealer filter), run.py (orchestrator)
web/         React dashboard (views/, components/, lib/)
data/        carpart_prices.csv (used engine/transmission prices) → seeds partsCosts
scripts/     CSV aggregation, car-brand dictionary build, local sandbox dispatcher
reference/    the three provided codebases reused per RULES #2
ARCHITECTURE.md the design every milestone conforms to
PROGRESS.md  the build loop's memory (milestones, reviews, overrides)
```

Operating notes & honest constraints

- **KSL and Facebook both restrict automated scraping.** The sandbox + (for FB) session- cookie approach is the practical workaround; expect to refresh the FB cookie and rotate if blocked. Retry/backoff and a "session expired" path are built in.
- **Valuation is only as good as the comps.** Every car shows its comp set and sample size so you can sanity-check; curve-valued cars are amber and never auto-alert.
- **Secrets** live in Convex env vars / untracked `.env.local`, never in the repo. The Daytona key is **env-var-only by design** — `settings.get` is a public query, so no secret belongs on the settings row. The FB cookie and MarketCheck key on the settings row are client-readable; keep the deployment single-tenant / not publicly exposed.
- **Alerts** are exactly-once per car (again only on a price drop). On a keyed deployment, delivery happens before the dedupe row commits, so a rare double-schedule could send twice though only one row lands; switch to claim-first before relying on it.
- **The pipeline** archives via the *Dead* stage (no row delete). **The app finds and ranks; you make the call and the contact** — it never messages sellers or moves money.